

IOT Project Runbook — Phase 4

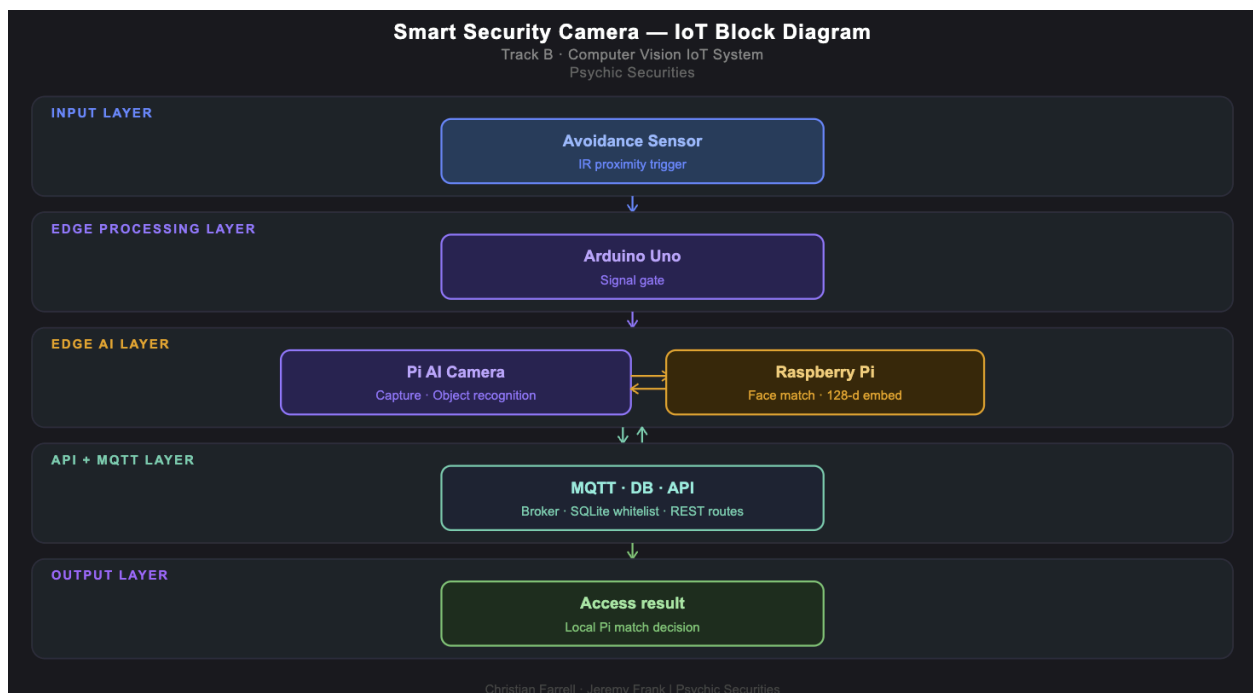
Smart Security Camera for Verified User Access

Track B — Computer Vision IoT System

Christian Farrell & Jeremy Frank

1. Architecture Diagram

This architecture diagram reflects an updated and simplified version of our original design. We removed the photoresistor sensor because it introduced an additional potential failure point without adding significant value to the core functionality of the system. The remaining hardware still supports the main workflow: detecting nearby activity, capturing an image, identifying whether a person is present, and checking that person against the authorized-user whitelist.



2. Hardware Used

The hardware for this project combines a Raspberry Pi, an Arduino, an avoidance sensor, and a Raspberry Pi AI Camera. The Raspberry Pi acts as the main edge computing device for the

system. It runs the Python services, manages the local SQLite whitelist database, publishes MQTT status/events, exposes the web dashboard and REST API, and performs the camera and face-recognition workflow when a trigger occurs. The Pi AI Camera provides the visual input for the system and supports on-device object detection through the IMX500 sensor, allowing the system to detect whether a person is present before attempting face recognition.

The Arduino is used as a lightweight sensor controller for the proximity side of the system. It reads the avoidance sensor, applies simple stability logic so noisy readings do not immediately trigger the camera, and sends structured JSON messages over serial to the Raspberry Pi. The avoidance sensor acts as the first-stage physical trigger, detecting when something is near the monitored area. Together, the Arduino and sensor reduce unnecessary camera processing by only notifying the Pi when nearby movement or obstruction is detected, while the Pi and AI Camera handle the higher-level verification and access decision.

3. Topic Names

home/security/alerts

Unknown-face alert notifications.

A typical payload contains event type, timestamp, confidence, and sensor identifier.

home/security/events

Operational/user events such as registration, unregister, and detection toggle.

home/security/status

Retained heartbeat/status snapshots including uptime, detection state, and component health.

4. How to Start The System

1. Confirm Prerequisites
 - Python 3.11+
 - 'uv' Python package manager
 - Mosquitto MQTT Broker Service (Mosquitto)
2. Wire the Arduino circuit:
 - a. (Avoidance/tracking sensor) -> (Arduino)
 - b. GND -> GND
 - c. +V+ -> 5V
 - d. OUT/S -> Digital 8
 - e. EN -> Digital 9 (Avoidance Sensor Only)
3. Push avoidance_serial.ino to the arduino and connect to the Pi.

4. Create a virtual environment:

```
> python -m venv .venv
```

5. Activate the virtual environment

```
> source .venv/bin/activate
```

6. Sync Dependencies

```
> uv sync
```

7. Create the environment file

```
> cp .env.example .env
```

8. Edit `.env` and set required values (at minimum API key, dashboard login, MQTT host/port)

9. Install Raspberry Pi Specific Dependencies

```
> uv sync -extra pi
```

10. Verify that your MQTT Broker is started (mosquitto)

```
mosquitto_sub -h localhost -t 'home/security/#' -v
```

11. Run the project

```
> uv run security-system
```

12. Open dashboard:

- URL: <http://127.0.0.1:5050/dashboard>
- Login: USER_EMAIL / USER_PASSWORD from .env

13. Perform a Quick Health Check

```
> curl http://127.0.0.1:5050/healthz
```

5. How to Run The Demo

Once you have successfully set up the project and see all components as 'UP' in the startup dashboard upon running it, we can proceed to running the project demo.

1. Start The System

```
> source .venv/bin/activate && uv run security-system
```

2. Confirm All Components are 'UP'

3. Open the Dashboard where it is Being Served (Below is the default location)

```
> dashboard available at http://127.0.0.1:5050
```

4. Disable the security system

5. Log in using your configuration in .env (USER_EMAIL, USER_PASSWORD)

6. Register a user; this will invoke the Camera, so be ready to smile!

7. If the user registers correctly, move on; if not, retry with a better angle or lighting.

8. If your Arduino sensor is properly set up and connected via Serial, skip to **Step 10**.

9. Run the 'fake-sensor' script if you do NOT have a live Arduino (Make sure to set SERIAL_PORT accordingly)

```
> uv run fake-sensor
```

10. Trigger your sensor (By pressing enter in the fake CLI) or by moving in front of the avoidance sensor. This should trigger a camera activation and an MQTT event.

11. Now repeat **Step 10** with an unregistered user; they should be denied.

12. Finally, unregister the registered user and repeat **Step 10** to see that they were now denied.

13. (Optional) Disable the MQTT, Camera, or Sensor Services to see that they are reflected in the status menu, re-enable them to see that they are added back.

14. Monitor your **artifacts/** directory to see the MQTT logs and captured images

6. Known Limitations

The avoidance sensor can sometimes be fooled if an object is placed in front of it and then slowly moved away. In that case, the sensor may continue acting as if it is obstructed, which can cause incorrect or delayed trigger behavior.

The system also has a limitation with multiple faces. If several faces are visible but none are recognized, the system only reports one unknown-face result using the highest similarity score, instead of creating separate unknown alerts for each face.

7. Fallback Plan

The two highest likelihoods of failure in our architecture originate from the MQTT service and the Sensor.

To ensure system resilience, we implemented failure detection and recovery mechanisms for both MQTT communication and sensor inputs.

For MQTT failures, if we detect that the component's status is not working or is disrupted, we have prepared a number of pre-existing backup artifacts that we can demo and explain. This ensures we can still effectively demo operations even when the primary data pipeline is unavailable.

For sensor failures, we developed a simulation approach to replicate sensor behavior. Specifically, we created a script (`arduino/fake-sensor.py`) that emulates a serial sensor connection. This script allows manual triggering of detection events, pressing "Enter" simulates incoming sensor data, enabling us to test system responses without requiring physical hardware.

In addition, we built a component status monitoring dashboard to provide visibility into overall system health. This dashboard tracks the status of key components, allowing us to quickly identify failures, verify fallback behavior, and ensure reliability across the system.